

Class 13: Transcriptomics and the analysis of RNA-Seq data

Mason Dinh (PID: A19140455)

Table of contents

Import Data	1
Toy differential gene expression	5
DESeq Analysis	10
Volcano plot	12
Save our results to date	16
Adding annotation data	16

Import Data

```
library(BiocManager)
```

```
Bioconductor version '3.22' is out-of-date; the current release version '3.23'
is available with R version '4.6'; see https://bioconductor.org/install
```

```
library(DESeq2)
```

```
Loading required package: S4Vectors
```

```
Loading required package: stats4
```

```
Loading required package: BiocGenerics
```

```
Loading required package: generics
```

Attaching package: 'generics'

The following objects are masked from 'package:base':

as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
setequal, union

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
unsplit, which.max, which.min

Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

findMatches

The following objects are masked from 'package:base':

expand.grid, I, unname

Loading required package: IRanges

Loading required package: GenomicRanges

Loading required package: Seqinfo

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats

Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

colAlls, colAnyNAs, colAnys, colAvgPerRowSet, colCollapse,
colCounts, colCummaxs, colCummins, colCumprods, colCumsums,
colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,
colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,
colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
colWeightedMeans, colWeightedMedians, colWeightedSds,
colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgPerColSet,
rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
rowWeightedSds, rowWeightedVars

Loading required package: Biobase

Welcome to Bioconductor

Vignettes contain introductory material; view with
'browseVignettes()'. To cite Bioconductor, see
'citation("Biobase")', and for packages 'citation("pkgname")'.

Attaching package: 'Biobase'

The following object is masked from 'package:MatrixGenerics':

rowMedians

The following objects are masked from 'package:matrixStats':

anyMissing, rowMedians

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG00000000003	723	486	904	445	1170
ENSG00000000005	0	0	0	0	0
ENSG000000000419	467	523	616	371	582
ENSG000000000457	347	258	364	237	318
ENSG000000000460	96	81	73	66	118
ENSG000000000938	0	0	1	0	2

	SRR1039517	SRR1039520	SRR1039521
ENSG00000000003	1097	806	604
ENSG00000000005	0	0	0
ENSG000000000419	781	417	509
ENSG000000000457	447	330	324
ENSG000000000460	94	102	74
ENSG000000000938	0	0	0

```
head(metadata)
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871

Q1. How many genes are in this dataset?

There are 38694 genes in this dataset

Q2. How many 'control' cell lines do we have?

```
sum(metadata$dex == "control")
```

```
[1] 4
```

Toy differential gene expression

```
colnames(counts) == metadata$id
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
ncol(counts)
```

```
[1] 8
```

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

- Find the “Control” columns in our `counts` object
- Extract just the “control” column values for all genes
- Calculate the average value per gene in these “control” columns

```
control.inds <- metadata$dex == "control"  
control.counts <- counts[,control.inds]  
control.mean <- rowMeans(control.counts)
```

Q3. How would you make the above code in either approach more robust? Is there a function that could help here?

To make the code above more robust, the control mean assumes that the `metadata$dex` is perfectly aligned with counts, so if a part of the sample is out of order or missing, then the calculated mean would be wrong. The more robust solution would be to use the `all()` command, which checks if every vector is TRUE.

Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called `treated.mean`)

```
treated.inds <- metadata$dex == "treated"
treated.counts <- counts[ ,treated.inds ]
treated.means <- rowMeans(treated.counts)
```

```
treated.mean <- rowMeans(counts[, metadata$dex == "treated"])
```

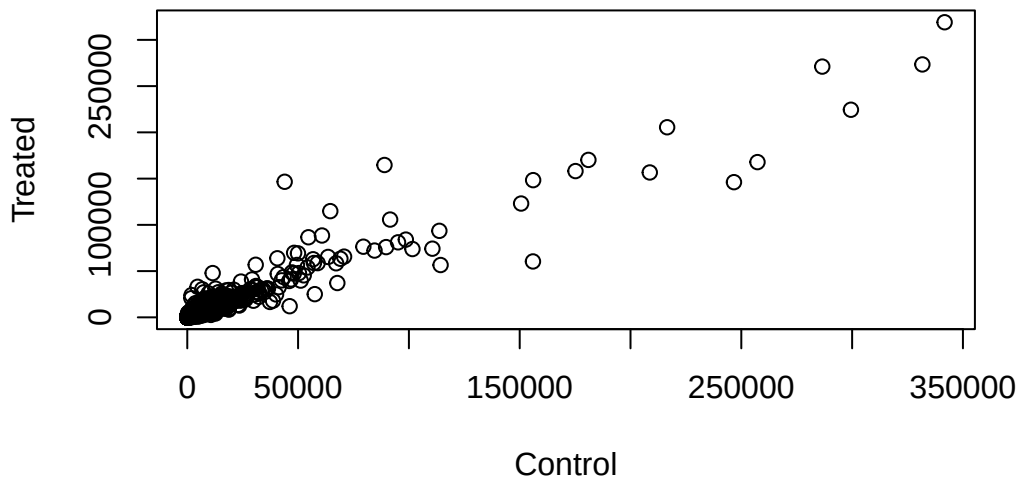
For book-keeping, let's store these together as a new object called `meancounts`

```
meancounts <- data.frame(control.mean, treated.mean)
head(meancounts)
```

	control.mean	treated.mean
ENSG000000000003	900.75	658.00
ENSG000000000005	0.00	0.00
ENSG000000000419	520.50	546.00
ENSG000000000457	339.75	316.50
ENSG000000000460	97.25	78.75
ENSG000000000938	0.75	0.00

Q5 (a). Create a scatter plot showing the mean of the treated samples against the mean of the control samples.

```
plot(meancounts[,1],meancounts[,2], xlab="Control", ylab="Treated")
```



Q5 (b). You could also use the ggplot2 package to make this figure producing the plot below. What geom_?() function would you use for this plot?

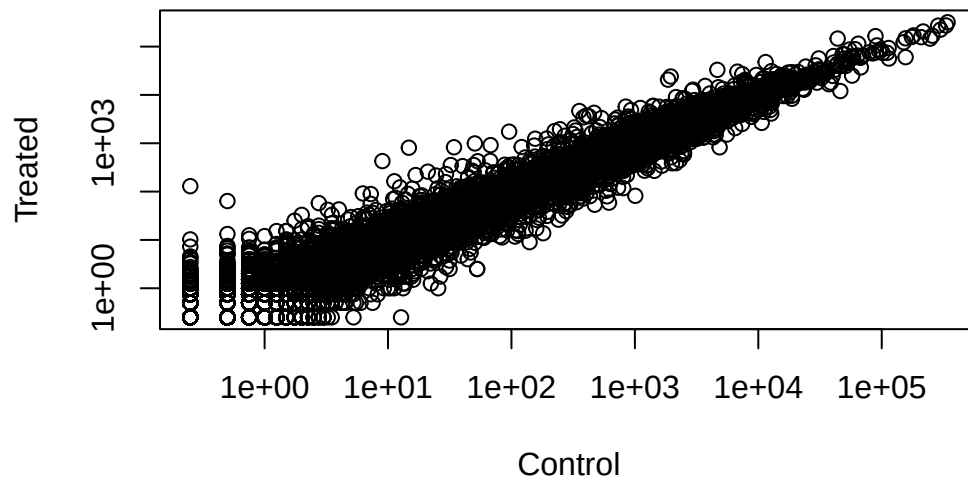
You would use the geom_point() function to make this type of plot with ggplot2.

Q6. Try plotting both axes on a log scale. What is the argument to plot() that allows you to do this?

```
plot(meancounts[,1], meancounts[,2], log="xy", xlab="Control", ylab="Treated")
```

Warning in xy.coords(x, y, xlabel, ylabel, log): 15032 x values <= 0 omitted from logarithmic plot

Warning in xy.coords(x, y, xlabel, ylabel, log): 15281 y values <= 0 omitted from logarithmic plot



We must often use $\log_2$ transform for this kind of data in bioinformatics because it makes my brain hurt less.

```
# Treated / Control
```

```
log2(20/20)
```

```
[1] 0
```

```
log2(40/20)
```

```
[1] 1
```

```
log2(10/20)
```

```
[1] -1
```

```
log2(80/20)
```

```
[1] 2
```


We call this little fraction the “log2 fold change” as it tells us how much more or less gene expression we have in units of doubling, etc.

Lets calculate the log2 fold change for our `treated.mean` and `control.mean` counts and call this `log2fc`.

```
meancounts$log2fc <- log2(meancounts$treated.mean / meancounts$control.mean)
head(meancounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000938	0.75	0.00	-Inf

A common “rule of thumb” threshold for calling a gene “up regulated” or “down regulated” is a log2 fold-change value of +2 or -2 (or greater)

Q7. What is the purpose of the `arr.ind` argument in the `which()` function call above? Why would we then take the first column of the output and need to call the `unique()` function?

The `arr.ind` argument determines how the `which()` function reports the location of values that meet the criteria of `==0`.

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)

to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000971	5219.00	6687.50	0.35769358
ENSG00000001036	2327.00	1785.75	-0.38194109

```
up.ind <- mycounts$log2fc > 2
down.ind <- mycounts$log2fc < (-2)
sum(up.ind)
```

```
[1] 250
```

```
sum(down.ind)
```

```
[1] 367
```

Q8. Using the up.ind vector above can you determine how many up regulated genes we have at the greater than 2 fc level?

There are 250 up regulated genes that are greater than 2 fc level.

Q9. Using the down.ind vector above can you determine how many down regulated genes we have at the greater than 2 fc level?

There are 367 down regulated genes that are greater than 2 fc level.

Q10. Do you trust these results? Why or why not?

No, these results should not be trusted because they do not determine whether the differences between groups is statistically significant, meaning that these results could be due to a lot of other variables or random noise.

DESeq Analysis

Let's do this analysis properly and not forget about the significance of the differences.

For this we will use the **DESeq2** package

```
library(DESeq2)
```

To run a DESeq analysis we need at least two inputs:

- countData i.e. are gene counts across different experiments
- colData i.e. our metadata about those count columns.

```
dds<- DESeqDataSetFromMatrix(countData = counts, colData = metadata, design = ~dex)
```

converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in design formula are characters, converting to factors

Now we can run DESeq analysis pipeline using this dds object that has

```
dds<-DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
res<-results(dds)
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029
	padj				
	<numeric>				
ENSG000000000003	0.163017				
ENSG000000000005	NA				
ENSG000000000419	0.175937				
ENSG000000000457	0.961682				
ENSG000000000460	0.815805				
ENSG000000000938	NA				

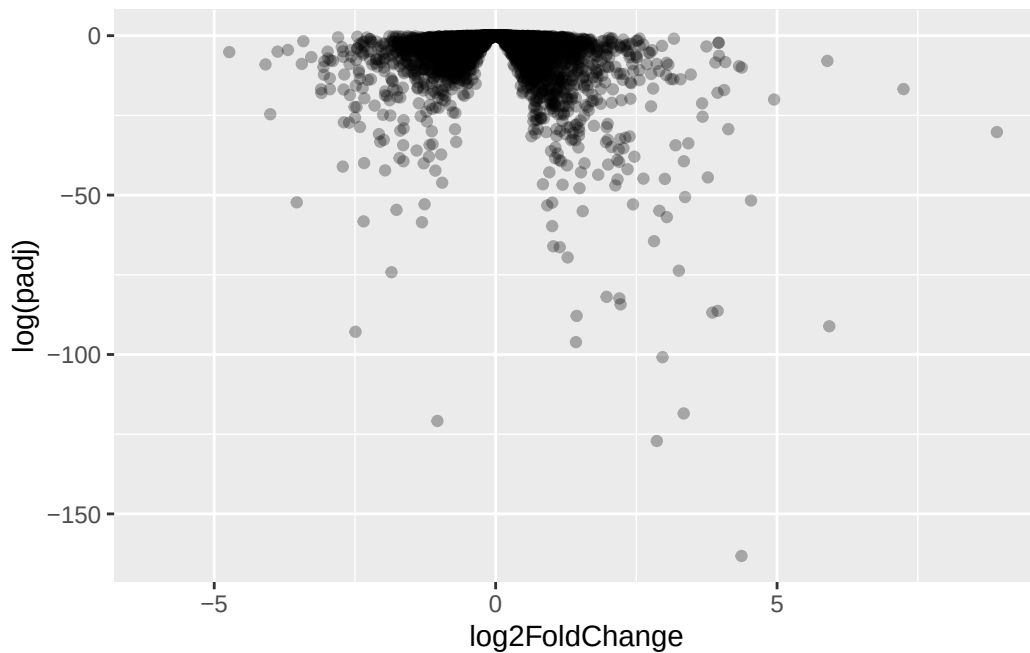
Volcano plot

This is a ubiquitous and common visualization for this type of data that puts the log2 foldchange and the adjusted p-value together in one plot that people can get insight for what is going on in the whole dataset results.

```
library(ggplot2)
```

```
ggplot(res) +  
  aes(log2FoldChange, log(padj)) +geom_point(alpha=0.3)
```

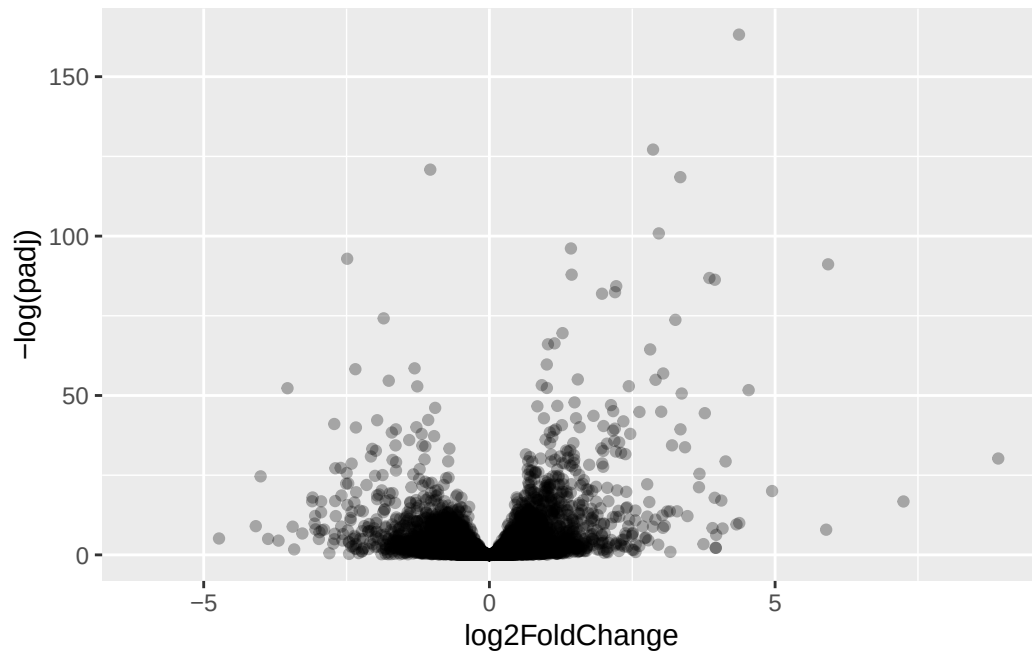
Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



That plot is not very useful because we don't care about genes with high p-values, we want the very low values below our alpha threshold.

```
ggplot(res) +  
  aes(log2FoldChange, -log(padj)) +geom_point(alpha=0.3)
```

Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



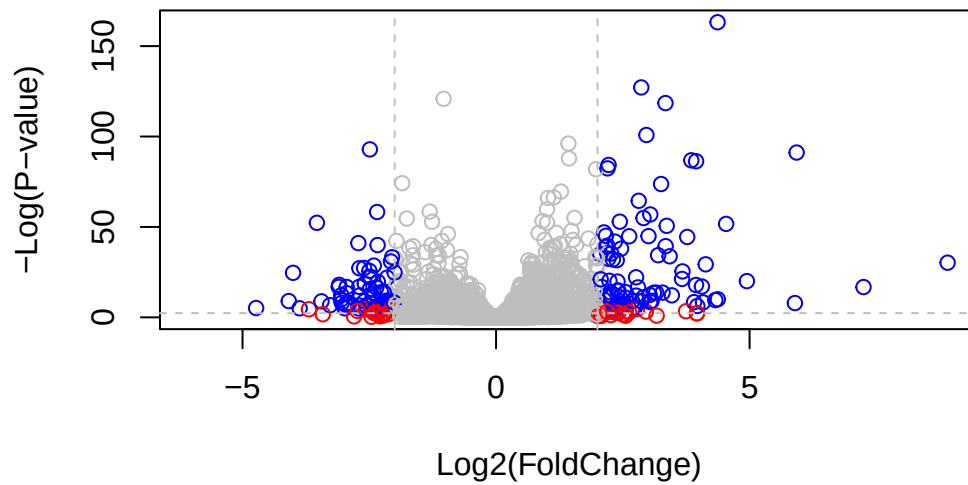
Q. Add annotations to this volcano plot including the log2 fold-change threshold of +2 and -2 and the p-value threshold of 0.05. Also color up just the genes that meet both these thresholds. These are the ones we will focus on next day!

```
mycols <- rep("gray", nrow(res))
mycols[ abs(res$log2FoldChange) > 2 ] <- "red"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2 )
mycols[ inds ] <- "blue"

plot( res$log2FoldChange, -log(res$padj),
      col=mycols, ylab="-Log(P-value)", xlab="Log2(FoldChange)" )

abline(v=c(-2,2), col="gray", lty=2)
abline(h=-log(0.1), col="gray", lty=2)
```



```
library("AnnotationDbi")
library("org.Hs.eg.db")
```

```
columns(org.Hs.eg.db)
```

[1]	"ACCNUM"	"ALIAS"	"ENSEMBL"	"ENSEMBLPROT"	"ENSEMBLTRANS"
[6]	"ENTREZID"	"ENZYME"	"EVIDENCE"	"EVIDENCEALL"	"GENENAME"
[11]	"GENETYPE"	"GO"	"GOALL"	"IPI"	"MAP"
[16]	"OMIM"	"ONTOLOGY"	"ONTOLOGYALL"	"PATH"	"PFAM"
[21]	"PMID"	"PROSITE"	"REFSEQ"	"SYMBOL"	"UCSCKG"
[26]	"UNIPROT"				

```
res$symbol <- mapIds(org.Hs.eg.db,
  keys=row.names(res),
  keytype="ENSEMBL",
  column="SYMBOL",
  multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
library(EnhancedVolcano)
```

Loading required package: ggrepel

```
x <- as.data.frame(res)

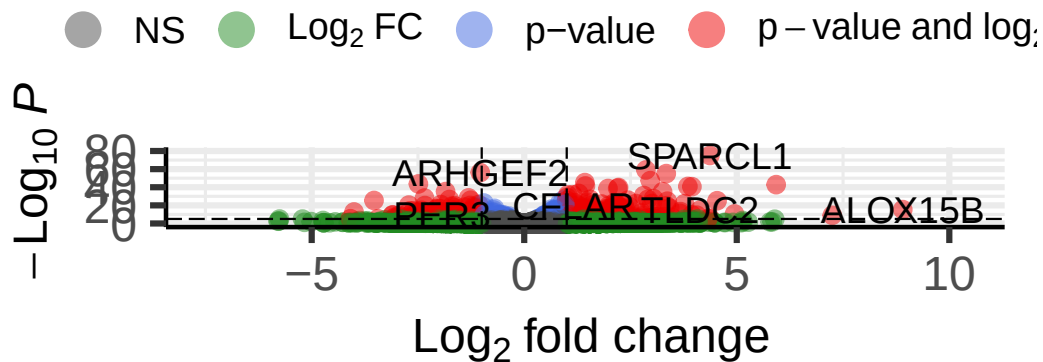
EnhancedVolcano(x,
  lab = x$symbol,
  x = 'log2FoldChange',
  y = 'pvalue')
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
i The deprecated feature was likely used in the EnhancedVolcano package.
Please report the issue to the authors.

Warning: The `size` argument of `element\_line()` is deprecated as of ggplot2 3.4.0.
i Please use the `linewidth` argument instead.
i The deprecated feature was likely used in the EnhancedVolcano package.
Please report the issue to the authors.

Volcano plot

EnhancedVolcano



total = 38694 variables

Save our results to date

```
write.csv(res,file="myresults.csv")
```

Adding annotation data

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 7 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029

	padj	symbol
	<numeric>	<character>
ENSG000000000003	0.163017	TSPAN6
ENSG000000000005	NA	TNMD
ENSG000000000419	0.175937	DPM1
ENSG000000000457	0.961682	SCYL3
ENSG000000000460	0.815805	FIRRM
ENSG000000000938	NA	FGR

Q11. Run the `mapIds()` function two more times to add the Entrez ID and UniProt accession and GENENAME as new columns called `res$entrez`, `res$uniprot` and `res$genename`.

```
res$entrez <- mapIds(org.Hs.eg.db,  
                     keys=row.names(res),  
                     column="ENTREZID",  
                     keytype="ENSEMBL",  
                     multiVals="first")
```

'select()' returned 1:many mapping between keys and columns


```
res$uniprot <- mapIds(org.Hs.eg.db,
                      keys=row.names(res),
                      column="UNIPROT",
                      keytype="ENSEMBL",
                      multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
res$genename <- mapIds(org.Hs.eg.db,
                       keys=row.names(res),
                       column="GENENAME",
                       keytype="ENSEMBL",
                       multiVals="first")
```

'select()' returned 1:many mapping between keys and columns

```
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.206107	0.101042	2.039828	0.0413675
ENSG000000000457	322.664844	0.024527	0.145134	0.168996	0.8658000
ENSG000000000460	87.682625	-0.147143	0.256995	-0.572550	0.5669497
ENSG000000000938	0.319167	-1.732289	3.493601	-0.495846	0.6200029
	padj	symbol	entrez	uniprot	
	<numeric>	<character>	<character>	<character>	
ENSG000000000003	0.163017	TSPAN6	7105	AOA087WYV6	
ENSG000000000005	NA	TNMD	64102	Q9H2S6	
ENSG000000000419	0.175937	DPM1	8813	H0Y368	
ENSG000000000457	0.961682	SCYL3	57147	X6RHX1	
ENSG000000000460	0.815805	FIRRM	55732	A6NFP1	
ENSG000000000938	NA	FGR	2268	B7Z6W7	
		genename			
		<character>			
ENSG000000000003		tetraspanin 6			

```

ENSG000000000005          tenomodulin
ENSG0000000000419 dolichyl-phosphate m..
ENSG0000000000457 SCY1 like pseudokina..
ENSG0000000000460 FIGNL1 interacting r..
ENSG0000000000938 FGR proto-oncogene, ..

```

```

ord <- order( res$padj )
#View(res[ord,])
head(res[ord,])

```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 10 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG00000152583	954.771	4.36836	0.2371306	18.4217	8.79214e-76
ENSG00000179094	743.253	2.86389	0.1755659	16.3123	8.06568e-60
ENSG00000116584	2277.913	-1.03470	0.0650826	-15.8983	6.51317e-57
ENSG00000189221	2383.754	3.34154	0.2124091	15.7316	9.17960e-56
ENSG00000120129	3440.704	2.96521	0.2036978	14.5569	5.27883e-48
ENSG00000148175	13493.920	1.42717	0.1003811	14.2175	7.13625e-46

	padj	symbol	entrez	uniprot
	<numeric>	<character>	<character>	<character>
ENSG00000152583	1.33157e-71	SPARCL1	8404	B4E2Z0
ENSG00000179094	6.10774e-56	PER1	5187	A2I2P6
ENSG00000116584	3.28806e-53	ARHGEF2	9181	A0A8Q3SIN5
ENSG00000189221	3.47563e-52	MAOA	4128	B4DF46
ENSG00000120129	1.59896e-44	DUSP1	1843	B4DRR4
ENSG00000148175	1.80131e-42	STOM	2040	F8VSL7

	genename
	<character>
ENSG00000152583	SPARC like 1
ENSG00000179094	period circadian reg..
ENSG00000116584	Rho/Rac guanine nucl..
ENSG00000189221	monoamine oxidase A
ENSG00000120129	dual specificity pho..
ENSG00000148175	stomatin

```

write.csv(res[ord,], "deseq_results.csv")

```